6. Implement Gradient descent & backpropagation in deep learning. neural network.

Aim:

To implement gradient descent and backpropagation algorithm in a simple deep neural network and study their role in training

Objectives:

1. To understand the working of gradient descent as an optimization method.
2. To implement backpropagation for updating weights in a neural network
3. To train a simple neural network for a classification task using these algorithms.
4. To observe how loss decreses with iterations

Pseudo Code

Start

1. Initialize dataset (x,y) for training (eg: XOR dataset)
2. Initialize neural network Parameters:
   - Input layer, hidden layer, output layer sizes
   - Random weights & biases
   - Learning rate (η)

3. Define forward Propagation

- $Z_1 = W_1 * X + b_1$
- A_1 = activation (z_1) # Sigmoid (or) ReLu
- $Z_2 = W_2 * A_1 + b_2$
- A_2 = activation (z_2) # Sigmoid / Softmax (output)

4. Compute loss using Mean Squared Error (or) Cross entropy

5, Backpropagtion:

- Compute error at output ($dA_2 = A_2 - y$)
- Calculate gradients for W_2, b_2

6, Update weigts and biases using gradient descent:

$$W = W - \eta * dw$$

$$b = b - \eta * db$$

7, Repeat the setps 3-6 for given number of epoches

8, Observe loss reduction and accuracy improvement

End.

Observation:

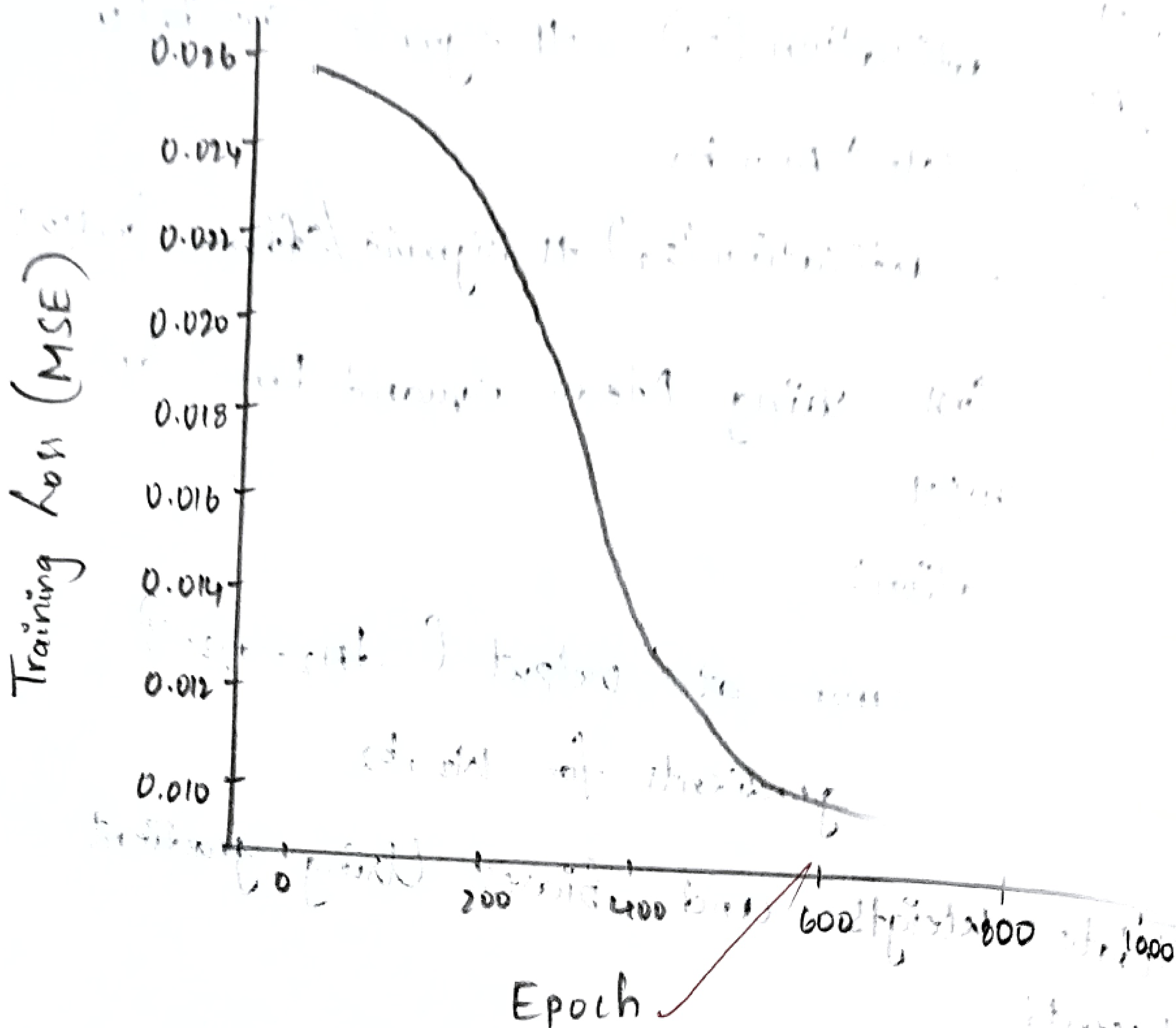
→ Loss decreases as the number of iterations (epochs) increases

→ Weights and biases adjust to minimize error

→ Backpropagation ensures errors are efficiently distributed layer by layer.

→ Learning rate (η) greatly influences convergence speed.

Training Loss over Epochs.



Epochs.

Epoch [100/1000] , Loss : 0.0071

Epoch [200/1000], Loss : 0.0070

Epoch [300/1000] , Loss : 0.0070

Epoch [400/1000] , Loss : 0.0069

Epoch [500/1000], Loss : 0.0069

Epoch [600/1000], Loss : 0.0068

Epoch [700/1000] , Loss : 0.0068

Epoch [800/1000], Loss : 0.0068

Epoch [900/1000], Loss : 0.0067

Epoch [1000/1000], Loss : 0.0066.

Result:

Successfully Implemented Gradient descent & backpropagation.